

EHU - Euskal Herriko Unibertsitatea

SAR: A Network File System

A Report for Activity 1.2

Distributed Systems

Authors: Baião, Giovana
Decubber, Dorien
Rodriguez Paredes, Roberto

Team: BDR

Teacher: Mikel Larrea Alava

Date: 21 September 2026

Contents

1 Introduction **1**

2 Description of the SAR/SZA service **1**

2.1 Architecture and components 1

2.2 Communication support 2

2.3 Functionality 2

3 Properties of a distributed file system **3**

4 Comparison with a cloud file service **4**

5 Conclusion **5**

Material used **5**

1 Introduction

A cloud file service hides where a file is stored and how it moves between devices. The user only sees local files; the service takes care of communication and storage in the background. Before looking at a system of that kind, it is useful to study a simpler one: a basic network file service.

This report analyses the file service used in the SAR/SZA subject. It is made of a client, a server, and a shared module that defines the protocol used between them. The client connects to the server and lets the user list, upload, download and delete files. These are already distributed operations, since the files live on a different machine and are reached only through the network.

The report tries to answer two questions. First, what support and functionality does the SAR/SZA service offer. Second, which properties of a distributed file system, as described in the course notes (Topics 1 and 4), does it meet, and what is still missing when compared with a cloud file service such as Dropbox.

2 Description of the SAR/SZA service

2.1 Architecture and components

The application follows a client–server architecture. The main pieces are:

- `cli_fich.py`: the client used to connect to the server and select an operation.
- `serv_fich.py`: the server that receives requests and accesses the file repository. Each connection is handled in a separate process.
- `serv_fich_multithread.py`: an alternative server that creates one thread per connection instead of one process.
- `szasar.py`: a module shared by client and server that defines the command names and the functions used to send and receive messages and file data.

Files are stored on the server's local file system. Depending on the server version, storage is organised either as a common directory shared by all users, or as a separate subdirectory per authenticated user. Both server variants can serve several clients at the same time, but neither implements an explicit coordination mechanism for two clients accessing the same file at once.

2.2 Communication support

Client and server communicate over TCP sockets, on a fixed port (6012). On top of TCP, `szasar.py` defines a small custom protocol: plain ASCII commands such as `USER`, `LIST` or `UPLO`, with `OK`/error replies, and binary transfer for the file content itself.

The protocol is stateful, in the sense that the server expects the commands in a fixed order.

2.3 Functionality

The service defines three fixed users: `anonimous`, `sar` and `sza`. The anonymous user can list and download files but cannot upload or delete them; the other two users need a password and get full access to the modifying operations. The server checks for unknown users, wrong passwords, missing files, requests sent in the wrong order, and failed transfers. Uploads are limited to 10 MiB, and the server always keeps at least 50 MiB of free space.

There is no notion of directories from the client's point of view: operations work directly on filenames. There is also no background synchronisation **(a file changed locally has no effect on the server copy until the user uploads it again)**.

Table 1 lists the commands defined by the protocol.

Command	Function	Description
`USER`	Identification	Sends the username and starts the login procedure.
`PASS`	Authentication	Sends the password for the selected user.
`LIST`	List files	Returns the available filenames and their sizes.
`DOWN / DOW2`	Download	Requests a file and confirms that the client is ready to receive it.
`UPLO / UPL2`	Upload	Requests an upload and then transfers the file content.
`DELE`	Delete	Removes a file from the server.
`EXIT`	End session	Closes the application session and the TCP connection.

Table 1: Commands provided by the SAR protocol

3 Properties of a distributed file system

Using the properties introduced in Topics 1 and 4 as a checklist, Table 2 summarises what the SAR/SZA service meets and what it lacks.

Property	Status	Key evaluation / limitations
Permanent storage	✓	Files persist on server disk across client disconnects.
Resource sharing	✓	Multi-client network access to centralized files.
File identification	▲	Flat filename lookup; lacks global or unique identifiers.
Access control	▲	Basic role checks (upload/delete); lacks per-file ACLs.
Concurrent access	▲	Parallel sessions supported, but no file locking mechanisms.
Access transparency	▲	Requires a dedicated client with explicit network commands.
Location transparency	✗	Requires explicit server address configuration.
Migration transparency	✗	Service or file relocation is not transparent to clients.
Replication transparency	✗	Single-copy architecture without replica coordination.
Failure transparency	✗	Network and server failures directly disrupt operations.
Scalability	✗	Centralized single-server design creates bottlenecks.
Availability	▲	Tied directly to single-node uptime; lacks redundancy.
Fault tolerance	✗	Single point of failure; no automatic failover or recovery.
Consistency	▲	Atomic overwrite on upload; no reconciliation mechanisms.
Security	▲	Credentials and payload transmitted in plaintext without TLS.

Status: ✓ Fulfilled ▲ Partial / constrained ✗ Not satisfied

Table 2: Evaluation of the SAR/SZA service

In short, SAR/SZA provides remote, shared, permanent storage with basic authentication, but it does not provide the properties that depend on replication – scalability, fault tolerance and full availability – since everything relies on a single server.

4 Comparison with a cloud file service

A cloud file service, such as Dropbox, is built to hide more of this distribution than SAR/SZA does. Table 3 gives a short, basic comparison, using the same visual layout as Table 2; it describes typical characteristics of a cloud service in general terms, not a detailed technical account of any single product.

Aspect	SAR/SZA	Cloud file service
User interaction	Explicit commands in a dedicated client	Normal local file operations with automatic synchronisation
Namespace	Flat or user-specific server directory, depending on the version	Structured folders and a consistent namespace across devices
Storage	One central server	Storage distributed or replicated across several machines
Failure handling	Operations stop when the server is unavailable	Redundancy and recovery reduce the effect of individual failures
Consistency	No conflict management between simultaneous clients	Synchronisation and version mechanisms manage updates and conflicts
Scalability	Limited by the capacity of one server	Capacity can be increased by adding storage and service nodes
Security	Basic authentication over unencrypted TCP	Encrypted communication and more detailed access control
Recovery	No version history or deleted-file recovery	Previous versions or deleted files may be recoverable
Offline use	No local synchronisation model	Local copies may remain usable and synchronise after reconnection

Table 3: SAR/SZA and cloud file service comparison

The gap comes mainly from replication and automatic synchronisation: a cloud service is built around several servers and a background process that keeps devices up to date, while SAR/SZA relies on one server and requires the user to trigger every transfer manually.

5 Conclusion

The SAR/SZA service is a basic network file service. It has a clear client–server structure, uses TCP for communication, and offers a small set of operations (login, list, upload, download, delete) built on a simple, custom protocol. This is enough to demonstrate remote resource sharing, which is the starting point of any distributed file system.

At the same time, measured against the properties in Topics 1 and 4, the service is clearly limited: there is no replication, no location transparency, and no policy for handling concurrent writers, and the single server is both a scalability bottleneck and a single point of failure. These are the aspects that separate a simple network file service from a full distributed or cloud file system.

Material used

- Distributed Systems, Topic 1: Introduction, UPV/EHU course notes.
- Distributed Systems, Topic 4: Distributed File Systems, UPV/EHU course notes.
- SAR/SZA protocol specification provided for the activity.
- Source files: `cli_fich.py`, `serv_fich.py`, `serv_fich_multithread.py`, `szasar.py`.